Java Technical MCQ — Round 2

Answer Key | 30 Questions | For 2 Years Experience | Duration: 45 min

CONFIDENTIAL — Interviewer Use Only

Test Overview

Total Questions: 40 | Categories: Strings, Core Java, OOP, Collections, Exception Handling, Java 8 Features, Design Patterns, Concurrency, Generics

Difficulty: Intermediate (2 Years Experience) | Topics: Core Java, OOP, Collections, Streams, Concurrency

Quick Answer Reference

Q1: B

Q2: C

Q3: B

Q4: B

Q5: B

Q6: B

Q7: B

Q8: C

Q9: B

Q10: C

Q11: B

Q12: A

Q13: B

Q14: B

Q15: C

Q16: C

Q17: B

Q18: B

Q19: B

Q20: B

Q21: C

Q22: A

Q23: C

Q24: C

Q25: B

Q26: A

Q27: C

Q28: B

Q29: B

Q30: B

Q31: B

Q32: A

Q33: B

Q34: B

Q35: A

Q36: B

Q37: B

Q38: A

Q39: B

Q40: C

Detailed Questions & Answers

STRINGS

Q1. What is the output of the following code?

```
String s1 = "Hello";
String s2 = "Hello";
String s3 = new String("Hello");
System.out.println(s1 == s2);
System.out.println(s1 == s3);
```

A. true, true

B. true, false '

C. false, false

D. Compilation Error

Explanation: s1 and s2 refer to the same String literal in the String pool, so == returns true. s3 creates a new String object on the heap, so s1 == s3 is false.

CORE JAVA

Q2. What is the output of the following code?

```
Integer a = 127;
Integer b = 127;
Integer c = 128;
Integer d = 128;
System.out.println(a == b);
System.out.println(c == d);
```

A. true, true

B. false, false

C. true, false '

D. false, true

Explanation: Java caches Integer objects for values -128 to 127. So a == b is true (same cached instance). 128 is outside the cache range, so c and d are separate heap objects — c == d is false.

OOP

Q3. What is the output of the following code?

```
class Animal {
    void sound() { System.out.println("Animal sound"); }
}
class Dog extends Animal {
    void sound() { System.out.println("Woof"); }
}
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
    }
}
```

A. Animal sound

B. Woof '

C. Compilation Error

D. Runtime Error

Explanation: This is runtime polymorphism (dynamic dispatch). Even though the reference type is Animal, the JVM calls the actual object's method — Dog.sound() — at runtime, printing "Woof".

OOP

Q4. What is the output of the following code?

```
class Parent {
    static void greet() { System.out.println("Hello from Parent"); }
}
class Child extends Parent {
    static void greet() { System.out.println("Hello from Child"); }
}
public class Main {
    public static void main(String[] args) {
        Parent obj = new Child();
        obj.greet();
    }
}
```

A. Hello from Child

B. Hello from Parent '

C. Compilation Error

D. Runtime Error

Explanation: Static methods are not overridden — they are hidden. Method resolution for static methods is done at compile time based on the reference type (Parent), not the runtime type. So Parent.greet() is called.

STRINGS

Q5. What is the output of the following code?

```
String s = "Java";
s.concat(" Programming");
System.out.println(s);
```

A. Java Programming

B. Java '

C. null

D. Compilation Error

Explanation: Strings are immutable. concat() returns a new String but does not modify the original. Since the return value is not assigned, s still points to "Java".

COLLECTIONS

Q6. What is the output of the following code?

```
import java.util.*;
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
list.remove(2);
System.out.println(list);
```

A. [1, 3, 4, 5]

B. [1, 2, 4, 5] '

C. [1, 2, 3, 4]

D. [1, 2, 3, 5]

Explanation: list.remove(2) calls remove(int index), removing the element at index 2 (value 3), not the Integer value 2. To remove by value, use list.remove(Integer.valueOf(2)).

COLLECTIONS

Q7. What is the output of the following code?

```
import java.util.*;
HashMap<String, Integer> map = new HashMap<>();
map.put(null, 1);
map.put(null, 2);
System.out.println(map.get(null));
```

A. 1

B. 2 '

C. null

D. NullPointerException

Explanation: HashMap allows exactly one null key. The second put(null, 2) overwrites the first. So map.get(null) returns 2.

EXCEPTION HANDLING

Q8. What happens when you compile the following code?

```
public class Test {
    public static void main(String[] args) {
        try {
            int[] arr = new int[5];
            arr[10] = 3;
        } catch (Exception e) {
            System.out.println("Exception");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Error");
        }
    }
}
```

A. Prints "Exception"

B. Prints "Array Error"

C. Compilation Error '

D. Prints both lines

Explanation: ArrayIndexOutOfBoundsException is a subclass of Exception. Catching a supertype before a subtype is unreachable code — the compiler throws an error: "exception ArrayIndexOutOfBoundsException has already been caught".

EXCEPTION HANDLING

Q9. What is the output of the following code?

```
public class Test {
    public static int getValue() {
        try {
            return 1;
        } finally {
            return 2;
        }
    }
    public static void main(String[] args) {
        System.out.println(getValue());
    }
}
```

A. 1

B. 2 '

C. Compilation Error

D. Throws an exception

Explanation: The finally block always executes, even after a return in try. When finally contains a return statement, it overrides the try block's return. The method returns 2.

JAVA 8 FEATURES

Q10. Which of the following is NOT a functional interface in Java 8?

A. java.lang.Runnable

B. java.util.Comparator<T>

C. java.io.Serializable '

D. java.util.concurrent.Callable<V>

Explanation: Serializable is a marker interface with no abstract methods and is not annotated with @FunctionalInterface. Runnable (run()), Comparator (compare()), and Callable (call()) are all valid functional interfaces.

JAVA 8 FEATURES

Q11. What is the output of the following code?

```
import java.util.*;
List<String> names = Arrays.asList("Charlie", "Alice", "Bob");
names.stream().sorted().forEach(System.out::println);
```

A. Charlie Alice Bob

B. Alice Bob Charlie '

C. Bob Alice Charlie

D. Compilation Error

Explanation: sorted() with no arguments sorts in natural (alphabetical) order. The output is Alice, Bob, Charlie — each on its own line.

JAVA 8 FEATURES

Q12. What is the output of the following code?

```
import java.util.*;
Optional<String> opt = Optional.of("Hello");
String result = opt.filter(s -> s.length() > 3).orElse("Default");
System.out.println(result);
```

A. Hello '

B. Default

C. null

D. NoSuchElementException

Explanation: "Hello" has length 5 > 3, so the filter predicate passes and the Optional still contains "Hello". orElse("Default") is only used when the Optional is empty. Output is "Hello".

CORE JAVA

Q13. If you override equals() in a class, what else MUST you also override to maintain the general contract?

A. toString()

B. hashCode() '

C. compareTo()

D. clone()

Explanation: The Java contract states: objects that are equal must have the same hashCode(). Overriding equals() without hashCode() breaks HashMap and HashSet behavior — equal objects may end up in different buckets.

DESIGN PATTERNS

Q14. In a thread-safe Singleton, what is wrong with the following implementation?

```
public class Singleton {
    private static Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
```

```
        return instance;
    }
}
```

A. The constructor should be public

B. It is not thread-safe — two threads can create two instances '

C. getInstance() should return void

D. Nothing is wrong — it is correctly implemented

> Explanation: This is a lazy singleton without synchronization. Two threads can both see instance == null simultaneously and each create a new instance. Fix by adding synchronized, using double-checked locking, or using an enum/holder pattern.

CONCURRENCY

Q15. What is the key difference between Thread.sleep() and Object.wait()?

A. sleep() can throw InterruptedException; wait() cannot

B. sleep() releases the object monitor lock; wait() does not

C. wait() releases the object monitor lock; sleep() does not '

D. Both release the lock and behave identically

> Explanation: Thread.sleep() pauses the current thread but keeps all locks. Object.wait() must be called inside a synchronized block and releases the monitor lock, allowing other threads to acquire it while waiting.

CONCURRENCY

Q16. What does the volatile keyword guarantee in Java?

A. Atomicity of compound operations like i++

B. Mutual exclusion — only one thread runs at a time

C. Visibility — changes to the variable are immediately visible to all threads '

D. Both atomicity and visibility

> Explanation: volatile guarantees visibility: reads/writes go directly to main memory, so all threads see the latest value. It does NOT guarantee atomicity — i++ (read-increment-write) is still a race condition even with volatile.

GENERICS

Q17. What type of wildcard does the following method signature use?

```
public static double sum(List<? extends Number> list) {
    double total = 0;
    for (Number n : list) total += n.doubleValue();
    return total;
}
```

A. Lower bounded wildcard (accepts Number and superclasses)

B. Upper bounded wildcard (accepts Number and subclasses) '

C. Unbounded wildcard (accepts any type)

D. This is invalid — wildcards cannot be used here

Explanation: ? extends Number is an upper bounded wildcard — it accepts Number itself or any subclass (Integer, Double, Long, etc.). Lower bounded would be ? super Number, and unbounded would be just ?.

COLLECTIONS

Q18. What is the output of the following code?

```
import java.util.*;
List<String> list = Arrays.asList("banana", "apple", "cherry");
Collections.sort(list);
System.out.println(list.get(0));
```

A. banana

B. apple '

C. cherry

D. UnsupportedOperationException

Explanation: Arrays.asList() returns a fixed-size but mutable list (elements can be set, not added/removed). Collections.sort() works by swapping elements via set(), so it succeeds. Sorted order: apple, banana, cherry. get(0) returns "apple".

STRINGS

Q19. What is the output of the following code?

```
StringBuilder sb = new StringBuilder("Hello World");
sb.delete(5, 11);
System.out.println(sb);
```

A. Hello World

B. Hello '

C. World

D. HelloWorld

Explanation: StringBuilder.delete(start, end) removes characters from index start (inclusive) to end (exclusive). Indices 5 through 10 are " World", so after deletion only "Hello" remains.

OOP

Q20. What is the output of the following code?

```
public class Test {
    static int x = 10;
    static {
        x = 20;
        System.out.println("Block: " + x);
    }
    public static void main(String[] args) {
        System.out.println("Main: " + x);
    }
}
```

A. Main: 10

B. Block: 20 Main: 20 '

C. Block: 10 Main: 10

D. Main: 20

Explanation: Static blocks execute when the class is loaded, before main(). The static block sets x = 20 and prints "Block: 20". Then main() runs and prints "Main: 20".

OOP

Q21. Which statement about abstract classes and constructors in Java is correct?

A. Abstract classes cannot have constructors

B. Abstract classes can have constructors and can be instantiated directly

C. Abstract classes can have constructors but cannot be instantiated directly '

D. Abstract classes can only have private constructors

Explanation: Abstract classes can define constructors. They are invoked implicitly via super() when a concrete subclass is instantiated. However, you cannot do new AbstractClass() directly — it causes a compilation error.

JAVA 8 FEATURES

Q22. What is the output of the following code?

```
interface A {
    default void print() { System.out.println("A"); }
}
interface B {
    default void print() { System.out.println("B"); }
}
class C implements A, B {
    public void print() { A.super.print(); }
}
new C().print();
```

A. A '

B. B

C. Compilation Error

D. A then B

Explanation: When a class inherits the same default method from two interfaces, it must override it. Inside the override, A.super.print() explicitly calls A's default implementation. Output is "A".

JAVA 8 FEATURES

Q23. What is the output of the following code?

```
import java.util.*;
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
int result = nums.stream().reduce(0, Integer::sum);
System.out.println(result);
```

A. 0

B. 5

C. 15 '

D. 120

Explanation: reduce(0, Integer::sum) starts with identity 0, then accumulates: 0+1=1, 1+2=3, 3+3=6, 6+4=10, 10+5=15. Output is 15.

CORE JAVA

Q24. Which of the following is a METHOD defined in java.lang.Object (not a keyword)?

A. final

B. finally

C. finalize '

D. All three are methods

Explanation: final and finally are Java keywords. finalize() is an instance method defined in java.lang.Object, historically called by the garbage collector before reclaiming memory (deprecated since Java 9).

EXCEPTION HANDLING

Q25. What interface must a class implement to be used in a try-with-resources statement in Java 7+?

A. java.io.Closeable only

B. java.lang.AutoCloseable '

C. java.lang.Disposable

D. java.lang.Resource

Explanation: The try-with-resources statement requires the resource class to implement java.lang.AutoCloseable (which declares close()). java.io.Closeable extends AutoCloseable, so Closeable resources also qualify.

CORE JAVA

Q26. What is the output of the following code?

```
enum Day { MON, TUE, WED, THU, FRI }
public class Test {
    public static void main(String[] args) {
        System.out.println(Day.WED.ordinal());
        System.out.println(Day.WED.name());
    }
}
```

A. 2, WED '

B. 3, WED

C. 2, wed

D. 3, THU

Explanation: ordinal() returns the 0-based position: MON=0, TUE=1, WED=2. name() returns the exact enum constant name as a String: "WED". Output: 2 then WED.

EXCEPTION HANDLING

Q27. Which of the following is a CHECKED exception that must be handled or declared?

A. NullPointerException

B. ArrayIndexOutOfBoundsException

C. IOException '

D. StackOverflowError

Explanation: IOException extends Exception (not RuntimeException), making it a checked exception — the compiler requires it to be caught or declared with throws. The others are either unchecked (RuntimeException subclasses) or Errors.

COLLECTIONS

Q28. What is the output of the following code?

```
import java.util.*;
List<String> original = new ArrayList<>(Arrays.asList("a", "b", "c"));
List<String> view = Collections.unmodifiableList(original);
original.add("d");
System.out.println(view.size());
```

A. 3

B. 4 '

C. UnsupportedOperationException

D. Compilation Error

Explanation: Collections.unmodifiableList() creates a read-only VIEW of the original list — not a copy. Modifications via original are reflected in view. Adding "d" to original changes view.size() to 4. Attempting to modify view directly would throw UnsupportedOperationException.

JAVA 8 FEATURES

Q29. Which of the following represents a static method reference in Java 8?

A. instance::methodName

B. ClassName::staticMethodName '

C. ClassName::new

D. super::methodName

Explanation: There are 4 types of method references: (1) Static: ClassName::staticMethod — e.g., Integer::parseInt; (2) Bound instance: obj::method; (3) Unbound instance: ClassName::instanceMethod; (4) Constructor: ClassName::new.

CORE JAVA

Q30. What is the output of the following Java 10+ code?

```
var numbers = new java.util.ArrayList<String>();
numbers.add("Java");
numbers.add("Python");
System.out.println(numbers.getClass().getSimpleName());
System.out.println(numbers.size());
```

A. Object, 2

B. ArrayList, 2 '

C. var, 2

D. Compilation Error

Explanation: var is local variable type inference (Java 10+). The compiler infers the type as ArrayList<String> at compile time — it is not dynamic typing. getSimpleName() returns "ArrayList" and size() returns 2.

COLLECTIONS

Q31. What is the output of the following code?

```
import java.util.*;
Map<String, Integer> map = new LinkedHashMap<>();
map.put("banana", 1);
map.put("apple", 2);
map.put("cherry", 3);
map.forEach((k, v) -> System.out.print(k + " "));
```

A. apple banana cherry

B. banana apple cherry '

C. cherry apple banana

D. Order is unpredictable

Explanation: LinkedHashMap preserves insertion order. Elements were inserted as banana !' apple !' cherry, so they are printed in that exact order. Use TreeMap for sorted order, HashMap for no guaranteed order.

CORE JAVA

Q32. What is the output of the following code?

```
int x = 5;
String result = (x > 3) ? "big" : (x > 1) ? "medium" : "small";
System.out.println(result);
```

A. big '

B. medium

C. small

D. Compilation Error

Explanation: The ternary operator evaluates left to right. x > 3 is true (5 > 3), so the result is immediately "big". The second ternary is never evaluated.

EXCEPTION HANDLING

Q33. What happens when the following code runs?

```
import java.util.*;
List<String> list = new ArrayList<>(Arrays.asList("a", "b", "c"));
for (String s : list) {
    if (s.equals("b")) list.remove(s);
}
```

A. Removes "b" successfully, list becomes [a, c]

B. Throws ConcurrentModificationException '

C. Throws IndexOutOfBoundsException

D. No exception — iterates without removing

Explanation: The enhanced for-loop uses an Iterator internally. Modifying the list directly while iterating invalidates the iterator's state, causing ConcurrentModificationException. Use Iterator.remove() or removeIf() instead.

JAVA 8 FEATURES

Q34. What is the output of the following code?

```
import java.util.*;
import java.util.stream.*;
List<String> words = Arrays.asList("hello", "world", "java");
String result = words.stream()
    .filter(w -> w.length() > 4)
    .collect(Collectors.joining(", "));
System.out.println(result);
```

A. hello, world, java

B. hello, world '

C. world, java

D. hello

Explanation: filter(w -> w.length() > 4) keeps strings with more than 4 characters: "hello" (5) and "world" (5). "java" (4) is excluded. Collectors.joining(", ") concatenates them with a comma-space separator.

CORE JAVA

Q35. What is the output of the following code?

```
Number n = Integer.valueOf(42);
System.out.println(n instanceof Integer);
System.out.println(n instanceof Double);
```

A. true, false '

B. false, true

C. true, true

D. Compilation Error

Explanation: The actual runtime object is an Integer (42). instanceof checks the runtime type of the object, not the reference type. Integer is a subclass of Number, so instanceof Integer is true. instanceof Double is false because the object is an Integer, not a Double.

JAVA 8 FEATURES

Q36. Which of the following statements about Java 8 interfaces is correct?

A. Interfaces still cannot have any method implementation

B. Interfaces can have static and default methods with full implementations '

C. A class uses the extends keyword to implement multiple interfaces

D. All interface methods are private by default in Java 8

Explanation: Java 8 introduced default methods (instance methods with a body) and static methods in interfaces. Classes still use implements (not extends). Interface methods are public and abstract by default unless declared default or static.

CORE JAVA

Q37. What is the key difference between Integer.parseInt("42") and Integer.valueOf("42")?

A. Both return a primitive int

B. parseInt() returns a primitive int; valueOf() returns an Integer object '

C. Both return an Integer object

D. valueOf() returns a primitive int; parseInt() returns an Integer object

Explanation: Integer.parseInt() returns a primitive int. Integer.valueOf() returns an Integer object (which may be cached for values -128 to 127). When auto-unboxing, both behave similarly, but the return types differ.

EXCEPTION HANDLING

Q38. What is the output of the following code?

```
try {
    String s = null;
    s.length();
} catch (NullPointerException | IllegalArgumentException e) {
    System.out.println("Caught: " + e.getClass().getSimpleName());
}
```

A. Caught: NullPointerException '

B. Caught: IllegalArgumentException

C. Compilation Error — cannot multi-catch these exceptions

D. Unhandled exception — program crashes

Explanation: The multi-catch syntax (Java 7+) catches multiple exception types in a single block. s.length() on null throws NullPointerException, which is caught. getSimpleName() returns "NullPointerException".

JAVA 8 FEATURES

Q39. What is the output of the following code?

```
import java.util.function.*;
Predicate<Integer> isEven = n -> n % 2 == 0;
Predicate<Integer> isPositive = n -> n > 0;
System.out.println(isEven.and(isPositive).test(-4));
System.out.println(isEven.or(isPositive).test(-4));
```

A. true, false

B. false, true '

C. false, false

D. true, true

Explanation: -4 is even (true) but not positive (false). and() requires both to be true — result is false. or() requires at least one to be true — -4 is even, so result is true. Output: false then true.

CONCURRENCY

Q40. Which statement about the following class is correct?

```
class Counter {
    private int count = 0;
    public synchronized void increment() { count++; }
    public int getCount() { return count; }
}
```

A. synchronized on increment() makes the entire class thread-safe

B. getCount() is also synchronized because count is a shared field

C. Multiple threads can safely call increment() without race conditions on that method '

D. The synchronized keyword here is redundant and has no effect

> Explanation: synchronized on increment() ensures only one thread executes that method at a time, preventing race conditions on count++. However, getCount() is NOT synchronized, so a thread calling getCount() may read a stale value — the class is not fully thread-safe.
